

STLRocket: Explainable Time Series Classification via STL Quantitative Semantics

Alessandro Della Siega

September 15, 2026

1 Introduction

STLRocket is a concept-based model for multivariate time series multi-class classification. Let Φ be a set of human-interpretable properties which can be satisfied or not by a time series. Such properties are Signal Temporal Logic Formulae (STL). Given a time series, the prediction depends only through the STL quantitative semantic of each concept in Φ :

$$f_{\Phi} : \mathcal{T} \rightarrow \mathcal{C}, \quad (1)$$

where $\mathcal{T}$ is the space of multivariate time series and $\mathcal{C} = \{1, \dots, K\}$ are the labels for K different class.

STLRocket is a ROCKET-inspired time series classifier. ROCKET applies to each time series a large number of random convolutional kernels; the resulting feature map is aggregated into 2 scalar values: the maximum value and the proportion of positive values. The classification task is then solved by training a multinomial logistic regression on the resulting features. STLRocket replaces random convolutional kernels with randomly generated STL formulae, and the aggregated feature is the untimed STL quantitative semantics. Classification is performed by training a LASSO multinomial logistic regression. This choice introduces two key aspects:

- STLRocket features are fully interpretable: an STL formula expresses a property of a time series, and the quantitative semantics express how much a trajectory satisfies such a property;
- Since features are interpretable and the classifier is interpretable, it is possible to build explanations for the predictions.
- The LASSO penalty induces sparsity in the learned coefficients, selecting a small subset of concepts that are most relevant for classification and discarding correlated features.

2 Preliminaries

3 Method

Let $\mathcal{D} = \{(\tau_i, y_i)\}_{i=1}^N$ be a dataset of labelled time series, indexes are partitioned in train and test set $\{1, \dots, N\} = \mathcal{I}_{\text{train}} \cup \mathcal{I}_{\text{test}}$ where $|\mathcal{I}_{\text{train}}| = N_{\text{train}}$. In particular, $\tau_i \in \mathbb{R}^{V \times T}$ is a V -dimensional time series which includes T equidistant time-steps. Each time series is classified by a label $y_i \in \mathcal{C}$.

3.1 Features

3.1.1 Formula Generation

The set of candidate formulae $\Phi = \{\phi_1, \dots, \phi_M\}$ is generated by recursive random sampling from the STL grammar

$$\phi ::= (x_v \leq \theta) \mid (x_v > \theta) \mid \neg \phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid G_{[a,b]} \phi \mid F_{[a,b]} \phi \mid \phi U_{[a,b]} \phi,$$

where x_v denotes the v -th state variable, $v \in \{1, \dots, V\}$, and G (Globally), F (Eventually), and U (Until) may be instantiated with an unbounded, bounded, or right-unbounded interval $[a, b]$, subject to the time horizon T of the signal. Each atomic predicate is generated by sampling a variable index v uniformly from $\{1, \dots, V\}$ and a threshold θ uniformly from the minimum and maximum value of the v -th state variable over the train set, with the direction of the inequality ($\leq$ or $>$) chosen uniformly at random.

Formulae are sampled recursively depth-first up to a maximum depth $d_{\max}$: at each recursion step of depth $d < d_{\max}$, the process either terminates at an atomic predicate, with probability growing linearly from a base rate $p_{\text{base}} = 0.05$ at $d = 0$ to 1 at $d = d_{\max}$, or expands into one of $\{\neg, \wedge, \vee, G, F, U\}$, sampled with the following weights $\{1, 1, 1, 1, 1, w_U\}$. The weight assigned to U is treated as a tunable parameter, denoted $w_U \in \{0, 1\}$, allowing the temporal operator Until to be included in or excluded from Φ . Recursion always terminates at $d = d_{\max}$, where the sampling process is forced to return an atomic predicate. The number of formulae $M = |\Phi|$, the maximum depth $d_{\max}$, and the Until weight w_U are hyperparameters of the model, and their effect on classification performance and formula complexity is assessed via ablation.

3.1.2 Feature Extraction

Given a fixed set of randomly generated STL formulae $\Phi = \{\phi_1, \dots, \phi_M\}$ and a time series τ_i , we define the following feature vector

$$x_i = \begin{pmatrix} \rho(\tau_i, \phi_1) \\ \vdots \\ \rho(\tau_i, \phi_M) \end{pmatrix} \in \mathbb{R}^M, \quad (2)$$

where $\rho(\tau_i, \phi_j) \in \mathbb{R}$ is the STL quantitative semantic evaluated at time $t = 0$. Intuitively, $x_{ij} > 0$ indicates that τ_i satisfies ϕ_j , while $x_{ij} < 0$ indicates violation; the magnitude reflects how robustly the formula is satisfied or violated.

Then, each feature is mean-centred and rescaled by standard deviation:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}, \quad (3)$$

where μ_j, σ_j are the empirical mean and empirical standard deviation of j -th feature, learnt on train set. The standardization of the features is needed for two reasons: improving the classifier training procedure and making the weights of different features comparable.

3.2 Classifier

The classifier is a LASSO multinomial logistic regression $g : \mathbb{R}^M \rightarrow \mathcal{C}$ that takes in input the standardized feature vector $\hat{x}_i$ and gives as output the predicted class $g(\hat{x}_i)$.

3.2.1 Multinomial Logistic Regression

The probability of $\hat{x}_i$ being classified with label $k \in \mathcal{C}$ is

$$P(y = k \mid \hat{x}_i) = \frac{\exp\left(\sum_{j=1}^M w_{k,j} \hat{x}_{i,j}\right)}{\sum_{l=1}^K \exp\left(\sum_{j=1}^M w_{l,j} \hat{x}_{i,j}\right)}, \quad (4)$$

where $\mathbf{W} = (w_{k,j}) \in \mathbb{R}^{K,M}$ is the matrix of the weights of the classifier. The label predicted for $\hat{x}_i$ is

$$g(\hat{x}_i) = \arg \max_{k \in \mathcal{C}} \sum_{j=1}^M w_{k,j} \hat{x}_{i,j}. \quad (5)$$

The intercept is not included in the model. Introducing the intercept would include a fixed per class offset that is not attributable to any STL formula and would therefore compete with the interpretable feature contributions for control of the decision, undermining the dependence of classification on the concepts in Φ .

3.2.2 Training

Let $y_i^{\text{OHE}} \in \{0, 1\}^K$ be the one-hot encoding of the true label y_i of τ_i in train set. The weights $\mathbf{W}$ are learnt by minimizing the LASSO-penalized negative log-likelihood

$$\text{nll}(\mathbf{W}) = -\frac{1}{N_{\text{train}}} \sum_{i \in \mathcal{I}_{\text{train}}} \sum_{k=1}^K y_{i,k}^{\text{OHE}} \log P(y_i = k \mid x_i) + \lambda \sum_{k=1}^K \sum_{j=1}^M |w_{k,j}|, \quad (6)$$

where $\lambda \geq 0$ is the regularization parameter chosen via cross-validation. [ADD WEIGHTS AND BALANCED ACCURACY SCORER FOR CV LAMBDA]

The minimization is performed using glmnet. This algorithm uses cyclical coordinate descent, which successively optimizes the objective function over each weight with others fixed, and cycles repeatedly until convergence.

3.2.3 Evaluation

Balanced accuracy is the metric used to assess the quality of the classifier over the test set.

3.3 Explanations

We aim to identify the STL formulae that drive the classification. We adopt a similar notion of *local explanations* introduced in Anchors, with the following adaptations:

- In our case feature predicates are STL formulae.
- In Anchors, a perturbation distribution is used to evaluate precision of a predicate on samples assumed to be similar to the one being explained. In our case, we leverage a feature space which is high dimensional by design, so a perturbation distribution on this space is almost certainly guaranteed to generate out-of-distribution samples. For this reason, the perturbation distribution is replaced by train data.
- In Anchors, a local explanation is a conjunction of feature predicates, constructed via beam search since the classifier is treated as a black box. In our setting, the classifier is interpretable, so we can build local explanations directly by inspecting its weights.

3.3.1 Local Explanations

Given a generic time series $\tau \in \mathbb{R}^{V \times T}$, its corresponding standardized feature vector $\hat{x} \in \mathbb{R}^M$, a classifier g , and the predicted label $k = g(\hat{x})$, a *local explanation* $\mathcal{L}(\tau)$ for such classification is an STL formula which is characterized by the following properties:

- Consistency:

$$\rho(\mathcal{L}(\tau), \tau) > 0, \quad (7)$$

- Precision:

$$\text{Prec}(\mathcal{L}(\tau)) > \alpha, \quad (8)$$

where

$$\text{Prec}(\mathcal{L}(\tau)) = \frac{\sum_{i \in \mathcal{I}_{\text{train}}} \mathbf{1}[k = y_i] \mathbf{1}[\rho(\mathcal{L}(\tau), \tau_i) > 0]}{\sum_{i \in \mathcal{I}_{\text{train}}} \mathbf{1}[\rho(\mathcal{L}(\tau), \tau_i) > 0]}, \quad (9)$$

and $\alpha \in [0, 1]$.

Another desirable property is coverage

$$\text{Cov}(\mathcal{L}(\tau)) = \frac{1}{N_{\text{train}}} \sum_{i \in \mathcal{I}_{\text{train}}} \mathbf{1}[\rho(\mathcal{L}(\tau), \tau_i) > 0]. \quad (10)$$

3.3.2 Formula Reparametrization

The following properties are needed to build explanations:

- $\rho(\neg\phi, \tau) = -\rho(\phi, \tau)$;
- $\rho(\phi, \tau) + \Delta = \rho(\phi_\Delta, \tau)$;

where $\Delta \in \mathbb{R}$ and ϕ_Δ is the formula ϕ in which every threshold has been shifted by a quantity whose magnitude is $|\Delta|$ and whose sign is chosen, atom by atom, so as to be consistent with the semantics of ρ : for an atom of the form $x \leq \theta$ (i.e. $\rho = \theta - x$) the threshold is updated as $\theta \mapsto \theta + \Delta$, while for an atom of the form $x > \theta$ (i.e. $\rho = x - \theta$) it is updated as $\theta \mapsto \theta - \Delta$; the sign is further flipped at every occurrence of a negation $\neg$ encountered while descending the formula tree, so that the identity above is preserved compositionally.

Reparametrization of an STL formula around a target class. Given a formula ϕ , we reparametrize ϕ so that its zero level-set (the boundary $\rho(\phi, \cdot) = 0$) is placed midway between the typical robustness of the target class k and that of the “closest” competing class, in the sense specified below.

Let

$$m_c = \text{median}(\{\rho(\phi, \tau_i) : y_i = c\}), \quad c \in \mathcal{C}, \quad (11)$$

denote the median robustness of ϕ over the time series of class c in the train set. Partition the remaining classes according to whether their median robustness lies below or above the median robustness of time series of target class m_k :

$$\mathcal{C}^- = \{c \neq k : m_c < m_k\}, \quad \mathcal{C}^+ = \{c \neq k : m_c > m_k\}. \quad (12)$$

The *reference class* r is selected as follows:

$$r = \begin{cases} \text{any } c \neq k, & \text{if } \mathcal{C}^- = \mathcal{C}^+ = \emptyset \text{ (single competing class),} \\ \arg \min_{c \in \mathcal{C}^+} m_c, & \text{if } \mathcal{C}^- = \emptyset, \\ \arg \max_{c \in \mathcal{C}^-} m_c, & \text{if } \mathcal{C}^+ = \emptyset, \\ \arg \min_{c \neq k} |m_c - m_k|, & \text{if } |\mathcal{C}^-| = |\mathcal{C}^+|, \\ \arg \max_{c \in \mathcal{C}^-} m_c, & \text{if } |\mathcal{C}^-| > |\mathcal{C}^+|, \\ \arg \min_{c \in \mathcal{C}^+} m_c, & \text{if } |\mathcal{C}^-| < |\mathcal{C}^+|. \end{cases} \quad (13)$$

Intuitively, the reference class is always the nearest competitor to m_k , so that the decision boundary is calibrated against the hardest competitor to separate; the cases differ only in which side of m_k is searched: the majority side when the competing classes are unevenly split around m_k , and both sides when the split is even.

If $m_k < m_r$, the formula is negated, $\phi \leftarrow \neg\phi$, and by the first property above the medians are updated as $m_k \leftarrow -m_k$, $m_r \leftarrow -m_r$; this guarantees, without loss of generality, that after this step $m_k \geq m_r$, i.e. that the (possibly negated) formula is oriented so as to be *satisfied* more strongly, on average, by the target class than by the reference class.

Finally, the formula is recentered by shifting all atom thresholds by

$$\Delta^* = -\frac{m_k + m_r}{2}, \quad (14)$$

yielding the reparametrized formula ϕ_{Δ^*} . By the second property above, the shift translates the robustness of every instance by the same additive constant Δ^* , and in particular the medians become

$$m_k \mapsto m_k + \Delta^* = \frac{m_k - m_r}{2}, \quad m_r \mapsto m_r + \Delta^* = -\frac{m_k - m_r}{2}, \quad (15)$$

i.e. the target and reference medians become symmetric around zero. Consequently, the zero-crossing of $\rho(\phi_{\Delta^*}, \cdot)$ is placed exactly halfway between the typical robustness of the target class and that of the reference class, making ϕ_{Δ^*} a calibrated local threshold for discriminating class k from its most relevant competitor r .

3.3.3 Building Local Explanations

The classification $k = g(\hat{x})$ can be equivalently stated as

$$\sum_{j=1}^M c_{l,j} > 0 \quad \forall l \in \mathcal{C} \setminus \{k\}, \quad (16)$$

where

$$c_{l,j} = (w_{k,j} - w_{l,j}) \hat{x}_j, \quad (17)$$

quantifies the contribution of the j -th feature in driving the classifier’s decision between class k and class l . If $c_{l,j} > 0$, the feature pushes the decision towards class k ; if $c_{l,j} < 0$, it works against the predicted class, pushing towards class l .

To each feature $\hat{x}_j$ is assigned the following class-contrastive score

$$s_j = \frac{1}{K} \sum_{l \neq k} c_{l,j} = (w_{k,j} - \bar{w}_j) \hat{x}_j, \quad (18)$$

where $\bar{w}_j = \frac{1}{K} \sum_{l=1}^K w_{l,j}$ and $j = 1, \dots, M$. The value of s_j can be interpreted as the alignment between $\rho(\tau, \phi_j)$ and the model's class-specific weighting of ϕ_j relative to its average weighting across all classes. To build a local explanation for the classification $k = g(\hat{x})$, we seek for formulae with high class-contrastive score and that satisfy the desired properties of local explanations.

Let $\mathcal{E}$ be the set which includes the reparametrized formulae selected by greedy selection via precision maximization among the top- p scoring formulae. Then, the local explanation is the conjunction of the formulae in $\mathcal{E}$. Formally, the local explanation for τ is

$$\mathcal{L}(\tau) = \bigwedge_{\phi \in \mathcal{E}} \tilde{\phi}, \quad (19)$$

where $\mathcal{E}$ is the set of selected formulae with $s_j > 0$ and $\tilde{\phi}$ is the reparametrization of formula ϕ around target class k .

3.3.4 Global Explanations

Given the local explanations for predictions based on on time series in in the train set, we can define a *global explanation* for class k as

$$\mathcal{G}(k) = \bigvee_{i \in \mathcal{I}_{\text{train}} : k = g(\hat{x}_i)} \mathcal{L}(\tau_i) \quad (20)$$

The metric used to assess the quality of $\mathcal{G}(k)$ is F_1 score.

3.4 Simplification of Global Explanations

4 Experiments

4.1 Datasets